Design and Analysis of Algorithm

Dr Suriya Prakash J
Department of CSE
JAIN (Deemed-to-be-University)

Algorithm Definition

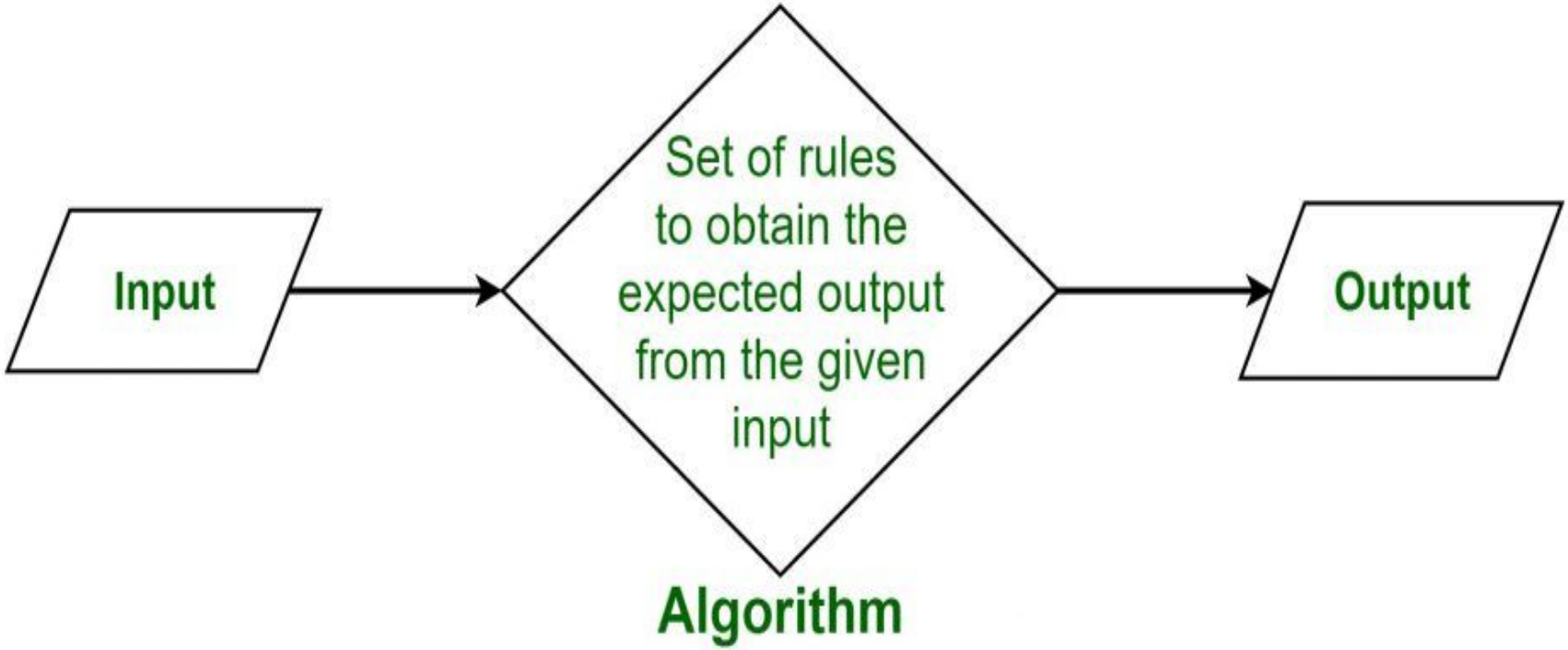
- *The word* ***Algorithm*** *means*

" A set of finite rules or instructions to be followed in calculations or other problem-solving operations "

Or

" A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations".

What is Algorithm?



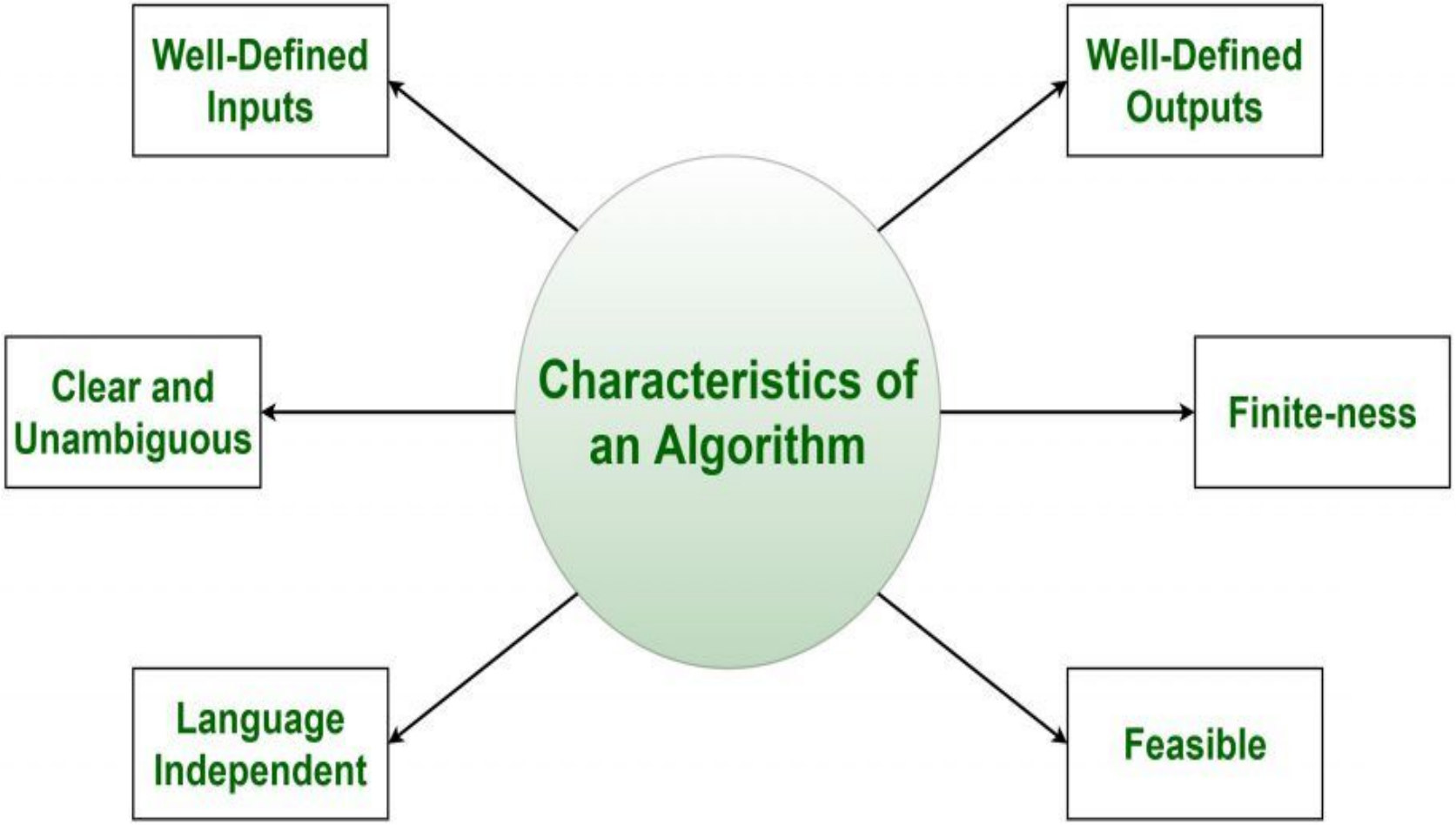
Use of the Algorithms

- **Computer Science:** Algorithms form the basis of computer programming and are used to solve problems ranging from simple sorting and searching to complex tasks.
- **Mathematics:** Algorithms are used to solve mathematical problems, such as finding the optimal solution to a system of linear equations or finding the shortest path in a graph.
- **Operations Research**: Algorithms are used to optimize and make decisions in fields such as transportation, logistics, and resource allocation.
- **Artificial Intelligence:** Algorithms are the foundation of artificial intelligence and machine learning, and are used to develop intelligent systems that can perform tasks such as image recognition, natural language processing, and decision-making.
- **Data Science:** Algorithms are used to analyze, process, and extract insights from large amounts of data in fields such as marketing, finance, and healthcare.

Need for algorithms

- Its necessary for solving complex problems efficiently and effectively.
- They help to automate processes and make them more reliable, faster, and easier to perform.
- Algorithms also enable computers to perform tasks that would be difficult or impossible for humans to do manually.
- They are used in various fields such as mathematics, computer science, engineering, finance, and many others to optimize processes, analyze data, make predictions, and provide solutions to problems.

Characteristics of an Algorithm



Characteristics of Algorithm

- **Clear and Unambiguous**: The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs**: If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.

Characteristics of Algorithm

- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Input**: An algorithm has zero or more inputs. Each that contains a fundamental operator must accept zero or more inputs.
- **Output**: An algorithm produces at least one output. Every instruction that contains a fundamental operator must accept zero or more inputs.
- **Definiteness:** All instructions in an algorithm must be unambiguous, precise, and easy to interpret. By referring to any of the instructions in an algorithm one can clearly understand what is to be done. Every fundamental operator in instruction must be defined without any ambiguity.
- **Finiteness:** An algorithm must terminate after a finite number of steps in all test cases. Every instruction which contains a fundamental operator must be terminated within a finite amount of time. Infinite loops or recursive functions without base conditions do not possess finiteness.
- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

Algorithm as a Technology

While the term "algorithm" is often associated with mathematics and computer science, it can be considered a fundamental concept and a powerful technology in its own right. An algorithm is a step-by-step procedure or set of rules for solving a particular problem or accomplishing a specific task. Algorithms are used not only in computer science but also in various other fields, and they play a crucial role in shaping and driving technological advancements. Here's how algorithms can be viewed as a technology:

- **Problem Solving:**
  - **Computational Problem Solving:** In computer science, algorithms are essential for solving computational problems. They provide a systematic approach to designing solutions for various tasks, from sorting data to searching for information.
- **Computing Technology:**
  - **Foundation of Computing:** Algorithms form the foundation of computing. They are the building blocks upon which software and applications are constructed. A well-designed algorithm can significantly impact the efficiency and performance of a computer program.
- **Data Processing:**
  - **Data Algorithms:** Algorithms are used for processing and analyzing data. Whether it's data sorting, searching, or performing complex computations on large datasets, algorithms are the driving force behind data-related technologies.
- **Artificial Intelligence (AI) and Machine Learning (ML):**
  - **Learning Algorithms:** In AI and ML, algorithms are used for training models and making predictions. Machine learning algorithms, in particular, enable systems to learn from data and improve their performance over time.
- **Optimization:**
  - **Optimization Algorithms:** Algorithms are widely used for optimization problems, such as finding the best solution among a set of possibilities. This is applicable in fields like operations research, logistics, and resource allocation.

- **Networking:**
  - **Routing Algorithms:** In networking, algorithms are employed to determine the most efficient paths for data transmission. Routing algorithms help manage the flow of data across networks.
- **Cryptography:**
  - **Encryption Algorithms:** Security technologies, such as encryption, rely on algorithms to protect sensitive information. Cryptographic algorithms ensure the confidentiality and integrity of data.
- **Biotechnology:**
  - **Bioinformatics Algorithms:** In biotechnology, algorithms are used for analyzing biological data, such as DNA sequences. Bioinformatics algorithms play a crucial role in genomics research.
- **Finance:**
  - **Financial Algorithms:** Algorithms are heavily utilized in financial technology (fintech) for tasks like risk assessment, fraud detection, and algorithmic trading.
- **Logistics and Planning:**
  - **Scheduling Algorithms:** In logistics and planning, algorithms are used for optimizing routes, scheduling tasks, and managing resources efficiently.
- **Human-Computer Interaction:**
  - **User Interface Algorithms:** Algorithms contribute to the design of user interfaces, helping create intuitive and responsive interactions between humans and computers.

Algorithm Analysis

- Analysis of efficiency of an **algorithm** can be performed at two different stages, before implementation and after implementation, as
- A priori analysis – This is defined as theoretical analysis of an algorithm. Efficiency of algorithm is measured by assuming that all other factors e.g. speed of processor, are constant and have no effect on implementation.
- A posterior analysis – This is defined as empirical analysis of an algorithm. The chosen algorithm is implemented using programming language. Next the chosen algorithm is executed on target computer machine. In this analysis, actual statistics like running time and space needed are collected.
- Algorithm analysis is dealt with the execution or running time of various operations involved. Running time of an operation can be defined as number of computer instructions executed per operation.

Algorithm Complexity

- Suppose X is treated as an algorithm and N is treated as the size of input data, the time and space implemented by the Algorithm X are the two main factors which determine the efficiency of X.
- Time Factor – The time is calculated or measured by counting the number of key operations such as comparisons in sorting algorithm.
- Space Factor – The space is calculated or measured by counting the maximum memory space required by the algorithm.
- The complexity of an algorithm f(N) provides the running time and / or storage space needed by the algorithm with respect of N as the size of input data.

Space Complexity

- **Space complexity of an algorithm** represents the amount of memory space needed the algorithm in its life cycle.
- Space needed by an algorithm is equal to the sum of the following two components
- A fixed part that is a space required to store certain data and variables (i.e. simple variables and constants, program size etc.), that are not dependent of the size of the problem.
- A variable part is a space required by variables, whose size is totally dependent on the size of the problem. For example, recursion stack space, dynamic memory allocation etc.
- Space complexity S(p) of any algorithm p is S(p) = A + Sp(I) Where A is treated as the fixed part and S(I) is treated as the variable part of the algorithm which depends on instance characteristic I. Following is a simple example that tries to explain the concept

Example For Space Complexity

Algorithm

- SUM(P, Q)
- Step 1 - START
- Step 2 - R ← P + Q + 10
- Step 3 – Stop
- Here we have three variables P, Q and R and one constant. Hence S(p) = 1+3. Now space is dependent on data types of given constant types and variables and it will be multiplied accordingly.

Time Complexity

- Time Complexity of an algorithm is the representation of the amount of time required by the algorithm to execute to completion. Time requirements can be denoted or defined as a numerical function t(N), where t(N) can be measured as the number of steps, provided each step takes constant time.

- For example, in case of addition of two n-bit integers, N steps are taken. Consequently, the total computational time is t(N) = c*n, where c is the time consumed for addition of two bits. Here, we observe that t(N) grows linearly as input size increases.

EXAMPLE

Imagine a classroom of 100 students in which you gave your pen to one person. You have to find that pen without knowing to whom you gave it.

- **$O(n^2)$:** You go and ask the first person in the class if he has the pen. Also, you ask this person about the other 99 people in the classroom if they have that pen and so on, This is what we call $O(n^2)$.

- **O(n):** Going and asking each student individually is O(N).

- **O(log n):** Now I divide the class into two groups, then ask: "Is it on the left side, or the right side of the classroom?" Then I take that group and divide it into two and ask again, and so on. Repeat the process till you are left with one student who has your pen. This is what you mean by O(log n).

Asymptotic Notations

- The main idea of asymptotic analysis is to have a measure of the efficiency of algorithms.
- Asymptotic Notations are mathematical tools that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows.
- Comparing space and time complexity using asymptotic analysis is possible.

Mainly three asymptotic notations

- ***Big-O Notation (O-notation)***

  O *Represent upper bound*

- ***Omega Notation (Ω-notation)***

  Ω *represents lower bound*

- ***Theta Notation (Θ-notation)***

  θ represents Average bound

Big-O Notation (O-notation)

$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \ldots < 2^n < 3^n < \ldots < n^n$

- ***Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.***
- *.It is the most widely used notation for Asymptotic analysis.*
  .It specifies the ***upper bound of a function.***
  .The maximum time required by an algorithm *or the* ***worst-case time complexity****.*
  .It returns the highest possible output value(big-O) for a given input.
  .Big-Oh(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

BIG-O Notation

If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that,

The function **f(n) ≤ O g(n)** iff +ve constant c and n0

Such that **f(n) ≤ c*g(n)** for all n ≥ n0

eg: f(n)=2n+3

2n+3 ≤ 2n+3n therefore 2n+3 ≤ 5n

apply n value as 2 then we will get

4+3 ≤ 4+6

7≤10 Therefore f(n)=O(n)

BIG-OMEGA Notation

If f(n) describes the running time of an algorithm, f(n) is **Ω**(g(n)) if there exist a positive constant C and n0 such that,

The function **f(n) ≥ Ω g(n)** iff +ve constant c and n0

Such that **f(n) ≥ c*g(n)** for all n ≥ n0

eg: f(n) =2n+3

2n+3 **≥** 1*n therefore 2n+3 **≥** n

apply n value as 2 then we will get

4+3 **≥** 2

7 **≥** 2 Therefore f(n)= **Ω**(n)

BIG-THETA Notation

If f(n) describes the running time of an algorithm, f(n) is **Ω**(g(n)) if there exist a positive constant C and n0 such that,

The function **f(n) ≤ θ g(n)** iff +ve constant c and n0

Such that **c1*g(n) ≤ f(n) ≤ c2*g(n)** for all n ≥ n0

eg: f(n) =2n+3

1*n**≤** 2n+3 **≤** 2n+3n therefore n**≤** 2n+3 **≤** 5n

apply n value as 2 then we will get

2**≤** 7 **≤** 10

Therefore f(n)= **θ**(n)